

UNIFECAP

Curso: Análise e Desenvolvimento de Sistemas

Disciplina: Engenharia de Software

Projeto: TechFlow Task Manager

Aluno: Daniel Santos Silva

Data: Julho/2026

1. Descrição do Projeto

Descrição

O **TechFlow Task Manager** é um sistema web desenvolvido em Python utilizando o framework Flask. O projeto tem como objetivo permitir o gerenciamento de tarefas através das operações básicas de CRUD (**Create, Read, Update e Delete**).

A aplicação possibilita ao usuário criar novas tarefas, visualizar tarefas cadastradas, editar informações existentes e excluir registros. O sistema possui uma interface simples e organizada, permitindo um gerenciamento eficiente das atividades.

O desenvolvimento do projeto aplicou conceitos de Engenharia de Software, desenvolvimento web, banco de dados, controle de versão e metodologias ágeis.

Escopo do Projeto

O escopo inicial do projeto consiste no desenvolvimento de uma aplicação web simples para gerenciamento de tarefas.

A aplicação foi desenvolvida utilizando o framework Flask, com armazenamento das informações em banco de dados SQLite, permitindo a implementação das principais funcionalidades de um sistema de gerenciamento:

- Cadastro de tarefas;
- Visualização de tarefas;
- Edição de tarefas;
- Exclusão de tarefas.

Além do desenvolvimento da aplicação, o projeto envolveu práticas de Engenharia de Software, como documentação, modelagem UML, controle de versões utilizando Git e GitHub, criação de testes automatizados e organização das atividades através da metodologia ágil Kanban.

2. Tecnologias Utilizadas

Para o desenvolvimento do **TechFlow Task Manager** foram utilizadas as seguintes tecnologias:

Python 3:

Linguagem de programação utilizada para desenvolvimento da aplicação.

Flask:

Framework web utilizado para criação da estrutura do sistema e gerenciamento das rotas.

HTML5 e CSS3:

Tecnologias utilizadas para construção e estilização da interface do usuário.

SQLite:

Banco de dados utilizado para armazenamento das informações das tarefas.

Git e GitHub:

Ferramentas utilizadas para controle de versão, armazenamento do código e acompanhamento do histórico de alterações.

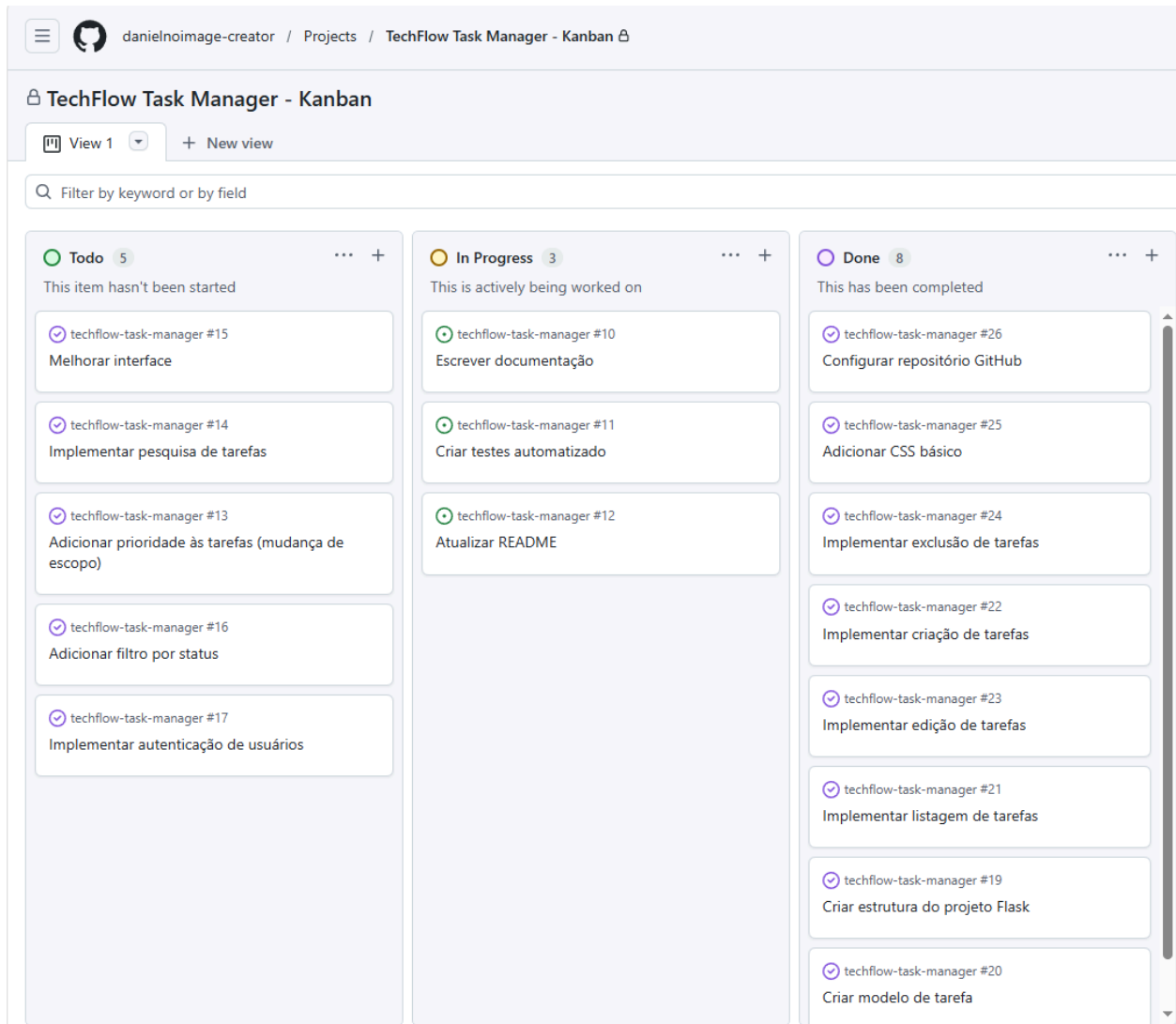
GitHub Projects:

Ferramenta utilizada para gerenciamento das atividades através do quadro Kanban.

Pytest:

Framework utilizado para criação e execução dos testes automatizados da aplicação.

3. Metodologia Ágil



Durante o desenvolvimento do projeto foi utilizada a metodologia ágil **Kanban**, organizada através do **GitHub Projects**.

O Kanban permitiu organizar as atividades do projeto de forma visual, facilitando o acompanhamento do desenvolvimento e o controle das tarefas.

O quadro foi dividido nas seguintes etapas:

To Do

Representa as tarefas planejadas que ainda não foram iniciadas.

In Progress

Representa as atividades que estão atualmente em desenvolvimento.

Done

Representa as funcionalidades finalizadas e validadas.

A utilização do Kanban contribuiu para uma melhor organização do projeto, permitindo acompanhar o progresso das funcionalidades e identificar quais atividades precisavam ser realizadas.

4. Importância da Modelagem de Software

A modelagem de software é uma etapa fundamental dentro da Engenharia de Software, pois permite representar visualmente a estrutura e o funcionamento de um sistema.

A utilização de modelos facilita o planejamento, desenvolvimento e manutenção da aplicação, permitindo que os desenvolvedores compreendam melhor os componentes do sistema antes da implementação.

Neste projeto foram utilizados diagramas UML para representar as funcionalidades e a estrutura da aplicação:

- Diagrama de Casos de Uso;
- Diagrama de Classes.

Esses diagramas auxiliaram na documentação do sistema e proporcionaram uma melhor compreensão da organização da aplicação.

5. Testes Automatizados

Foram implementados testes automatizados utilizando o framework **Pytest**, com o objetivo de verificar o funcionamento correto das principais funcionalidades do sistema.

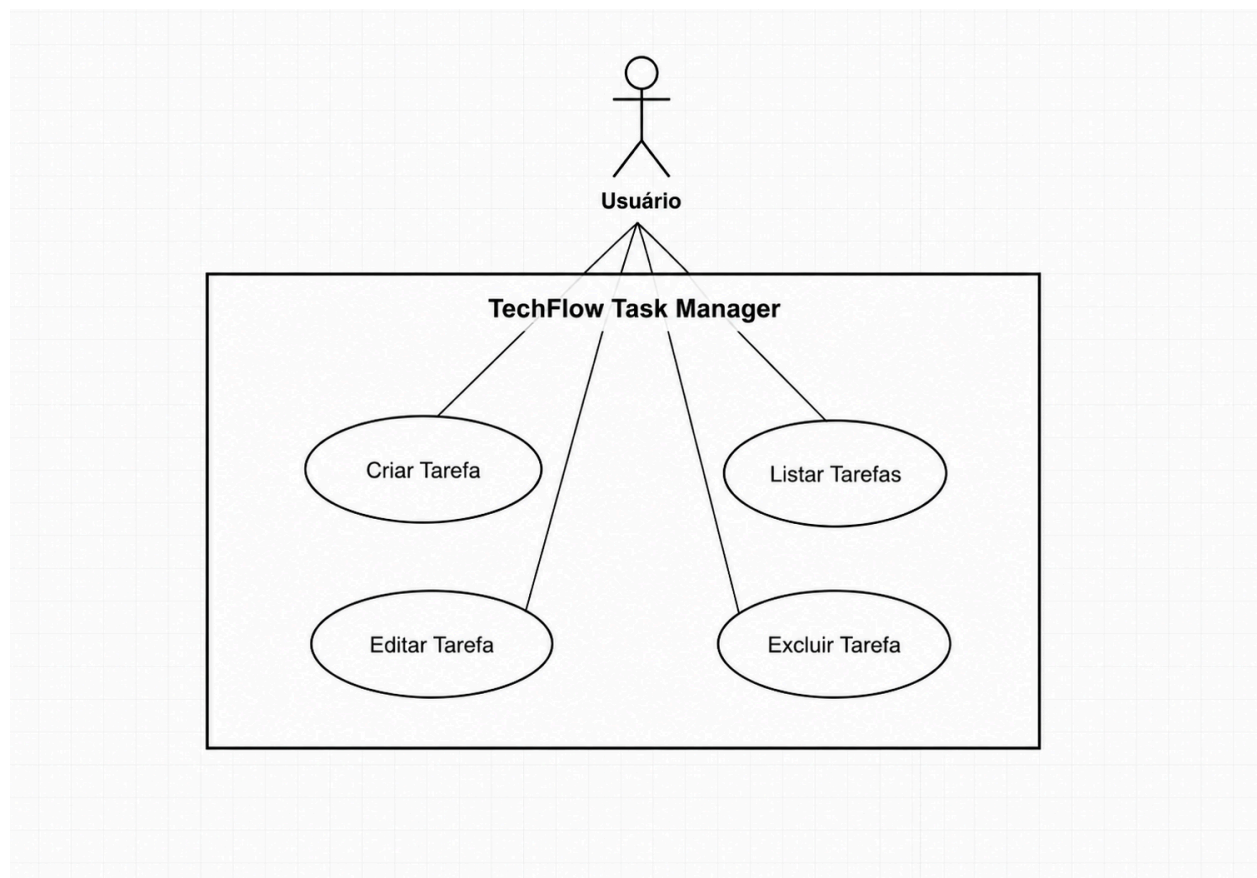
Os testes desenvolvidos validam se as páginas principais da aplicação são carregadas corretamente, garantindo maior confiabilidade durante o desenvolvimento.

Foram realizados testes para verificar:

- Carregamento da página inicial;
- Carregamento da página de criação de tarefas.

A utilização de testes automatizados permite identificar possíveis problemas de forma rápida e facilita futuras alterações no código.

6. Diagrama de Casos de Uso



O Diagrama de Casos de Uso representa a interação entre o usuário e o sistema.

Ele apresenta as principais funcionalidades disponíveis no **TechFlow Task Manager**, permitindo visualizar as ações que podem ser realizadas pelo usuário.

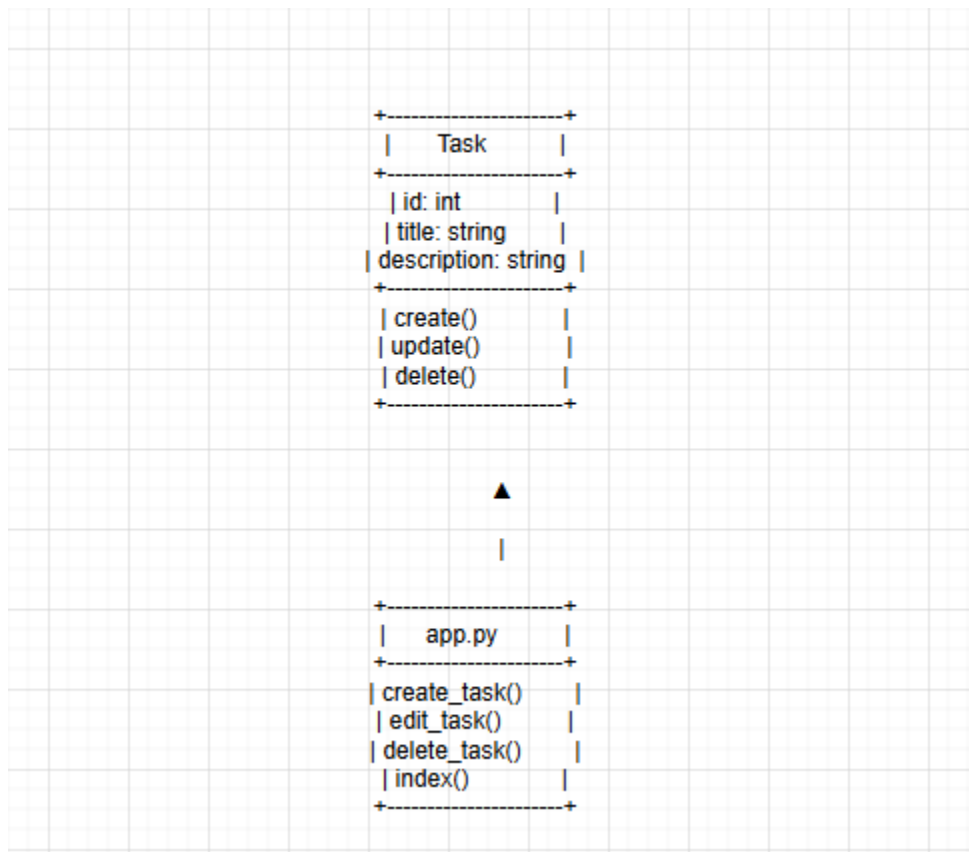
As principais funcionalidades representadas são:

- Criar tarefa;
- Listar tarefas;
- Editar tarefa;
- Excluir tarefa.

O diagrama auxilia na compreensão dos requisitos funcionais da aplicação.

(Inserir aqui a imagem do Diagrama de Casos de Uso)

7. Diagrama de Classes



O Diagrama de Classes apresenta a estrutura interna do sistema, demonstrando os principais elementos utilizados durante o desenvolvimento.

O principal componente representado é a classe **Task**, responsável pelo armazenamento das informações das tarefas cadastradas.

São apresentados seus principais atributos e relacionamentos com a aplicação, permitindo compreender a organização do código e a estrutura dos dados utilizados.

(Inserir aqui a imagem do Diagrama de Classes)

8. Controle de Versão e Desenvolvimento

Durante o desenvolvimento foi utilizado o Git e o GitHub para controle de versão do projeto.

Foram realizados commits seguindo o padrão de mensagens semânticas (**Semantic Commits**), facilitando a identificação das alterações realizadas.

Exemplos de commits realizados:

- docs: adiciona diagramas UML
- test: melhora documentação dos testes

Essa organização possibilitou manter um histórico claro das modificações realizadas no projeto.

9. Conclusão

O desenvolvimento do **TechFlow Task Manager** possibilitou aplicar conhecimentos adquiridos na disciplina de Engenharia de Software, envolvendo desenvolvimento web, programação em Python utilizando Flask, banco de dados, testes automatizados, documentação e metodologia ágil.

A utilização do Kanban auxiliou na organização das atividades, enquanto os diagramas UML contribuíram para uma melhor compreensão da estrutura e funcionamento do sistema.

O projeto demonstrou a importância de aplicar boas práticas de desenvolvimento, controle de versão e documentação para criação de softwares mais organizados, confiáveis e fáceis de manter.